# Structures with C++

DSA using C++

# What is a structure?

A structure is a user-defined data type in C/C++. A structure is a collection of variables of different data types under a single name. A structure creates a data type that can be used to group items of possibly different types into a single type.

**Why Structures?**

**Example 1:**

How would you represent student details like: id, name, school, level, group?

- You can easily create different variables ***id, name, school, level and group*** to store these information separately.
- What will happen if you want to store information about multiple students. Now, you need to create different variables for each information per student:
  ***id1, name1, school1, level1, group1,i2d, name2, school1, level2, group2,...***
  It is clear that you will have many lines of code which is a bad practice.

A better approach will be to have a collection of all related information under a single name **Student**, and use it for every student. Now, the code looks much cleaner, readable and efficient as well.

This collection of all related information under a single name Student is a structure.

# Structure Members

Structures in C++ can contain two types of members:

- **Data Member**: These members are normal C++ variables. We can create a structure with variables of different data types in C++.
- **Member Functions**: These members are normal C++ functions. Along with variables, we can also include functions inside a structure declaration.

**Example:**

```
struct Student{
// Data Members
int id;
int age;
int marks;

// Member Functions
void printDetails()
{
        cout<<"Roll = "<<id<<"\n";
        cout<<"Age = "<<age<<"\n";
        cout<<"Marks = "<<marks;
}
};
```

In the above structure, the data members are three integer variables to store *id number, age and marks* of any student and the member function is *printDetails()* which is printing all of the above details of any student.

Rwanda Coding Academy

## Declare a structure in C++

The `struct` keyword defines a structure type followed by an identifier (name of the structure). Then inside the curly braces, you can declare one or more members (declare variables inside curly braces) of that structure. For example:

```
struct Student{
int id;  // 32
string name; //"Mugabo Javis"
string school; // "Rwanda Coding Academy"
string level;   // "Year2"
char group; // 'A' or ; 'B', 'C'
};
```

Here the structure Student is declared with 5 members.

**Note**:  The declaration of the structure must end with a semicolon.

Rwanda Coding Academy

## Declare a structure in C++

The `struct` keyword defines a structure type followed by an identifier (name of the structure). Then inside the curly braces, you can declare one or more members (declare variables inside curly braces) of that structure. For example:

```
struct Student{
int id;  // 32
string name; //"Mugabo Javis"
string school; // "Rwanda Coding Academy"
string level;   // "Year2"
char group; // 'A' or ; 'B', 'C'
};
```

Here the structure Student is declared with 5 members.

**Note**:  The declaration of the structure must end with a semicolon.

*When a structure is created, no memory is allocated.*
*The structure definition is only the blueprint for the creation of variables.*
*You can imagine it as a datatype.*
*When you define an integer as: int age;*
*The `int` specifies that, variable `age` can hold integer element only.*
*Similarly, structure definition only specifies that, what property a structure variable holds when it is defined.*

Rwanda Coding Academy

A structure variable can either be declared with structure declaration or as a separate declaration like basic types as follow:

```
// A variable declaration with structure
declaration.
struct Student{
int id;
string name;
string school;
string level;
char group;
}student1;

// The variable student is declared with
'Student'
```

```
// A variable declaration like basic data
types
struct Student{
int id;
string name;
string school;
string level;
char group;
};
int main(){
 struct Student student1;
 // The variable student1 is declared like
a norma
}
```

***Note:*** *In C++, the struct keyword is optional before in declaration of a variable. In C, it is mandatory.*

# A structure with many variables

To declare more than one structure variable we separate them using comma as follow:

Example 1:

```
struct Student{
int id;
string name;
string school;
string level;
char group;
}student1, student2, student2;
```

Example2:

```
struct Point{
int x,y;
}p1, p2;
```

Rwanda Coding Academy

# Access members of a Structure

The members of structure variable is accessed using a dot **(.)** operator. Suppose, you want to access `members of` of structure variable `student`. You can perform this task by using following code below:

```
Student student;

student.name="Ishimwe Mary";
student.id=32;
student.school="RCA";
student.district="Nyabihu";
student.level="Year2";
student.group='A';
```

**Alternatively, you can assign member variables directly to a structure:**

```
Student  mj={“Mugabo Javis”, 32,”RCA”,”Year2”, ’A’};
```

Rwanda Coding Academy

# Access members of a Structure

C++ Program to assign data to members of a structure variable and display it. Below we are going to read student details and display them to the console:

```
#include <iostream>
using namespace std;
struct Student{
int id;
string name;
string school;
string level;
char group;
};
```

```
int main()
{
  Student student;
    cout << "Enter i: ";
    cin>>student.id;
    cin.ignore();
    cout << "Enter name: ";
    getline(cin,student.name);
    cout << "Enter school: ";
  getline(cin,student.school);
cout << "Enter level: ";
    getline(cin,student.level);
cout << "Enter group: ";
    cin>>student.group;
```

```
   cout<<"Display student
Information:"<<endl;
cout<<student.id<<endl;
cout<<student.name<<endl;
cout<<student.school<<endl;
cout<<student.level<<endl;
cout<<student.group<<endl;
return 0;
}
```

Rwanda Coding Academy

# Structure and Function:Passing structure to function in C++

Structure variables can be passed to a function and returned in a similar way as normal arguments.
Consider using a function to print a student in  this example:

```
#include <iostream>
using namespace std;
struct Student{
int id;
string name;
string school;
string level;
char group;
};
void print_student(Student s){
cout<<s.id<<endl;
cout<<s.name<<endl;
cout<<s.school<<endl;
cout<<s.level<<endl;
cout<<s.group<<endl;
```

```
int main()
{
  Student student;
    cout << "Enter i: ";
    cin>>student.id;
    cin.ignore();
    cout << "Enter name: ";
    getline(cin,student.name);
    cout << "Enter school: ";
  getline(cin,student.school);
cout << "Enter level: ";
    getline(cin,student.level);
cout << "Enter group: ";
    cin>>student.group;
```

```
   cout<<"Display student
Information:"<<endl;
print_student(student);
return 0;
}
```

# Structure and Function:Passing structure to function in C++

Structure variables can be passed to a function and returned in a similar way as normal arguments.
Consider using a function to make a student then print a student in this example:

```
#include <iostream>
using namespace std;
struct Student{
int id;
string name;
string school;
string level;
char group;
};
void print_student(Student s){
cout<<s.id<<endl;
cout<<s.name<<endl;
cout<<s.school<<endl;
cout<<s.level<<endl;
cout<<s.group<<endl;
}
```

```
Student make(Student mj){
mj.name="Mugabo Javis";
mj.id=32;
mj.school="RCA";
mj.level="Year2";
mj.group='A';
return  mj;
}
```

```
  int main()
{
  Student st;
  Student s=make(st);
cout<<"Display student
Information:"<<endl;
print_student(s);
return 0;
}
```

# Pointer to  Structure

A pointer variable can be created not only for native types like (int, float, double etc.) but they can also be created for user defined types like structure. Here is how you can create pointer for structures:

```
#include <iostream>
using namespace std;
struct Student{
int id;
string name;
string school;
string level;
char group;
};
```

```
int main()
{
  Student *student, st;
student=&st;
    cout << "Enter i: ";
    cin>>(*student).id;
    cin.ignore();
    cout << "Enter name: ";
    getline(cin,(*student).name);
    cout << "Enter school: ";
  getline(cin,(*student).school);
cout << "Enter level: ";
    getline(cin,(*student).level);
cout << "Enter group: ";
    cin>>(*student).group;
```

```
  cout<<"Display student
Information:"<<endl;
cout<<(*student).id<<endl;
cout<<(*student).name<<endl;
cout<<(*student).school<<endl;
cout<<(*student).level<<endl;
cout<<(*student).group<<endl;
return 0;
}
```

Rwanda Coding Academy

# Pointer to Structure(next)

In this program, a pointer variable `student` and normal variable `st` of type structure `Stutdent` is defined.

The address of variable **`st`** is stored to pointer variable, that is, `student` is pointing to variable `st`. Then, the member function of variable **`st`** is accessed using pointer.

**Notes:**

- Since pointer `ptr` is pointing to variable `d` in this program, `(*student).id` and `st.id` are equivalent. Similarly, `(*ptr).name` and st.name are equivalent.
- However, if we are using pointers, it is far more preferable to access struct members using the `->` operator, since the `.` operator has a higher precedence than the `*` operator.

  Hence, we enclose `*student` in brackets when using `(*student).name, etc`. Because of this, it is easier to make mistakes if both operators are used together in a single code. See example below:

```
cout<<"Display student Information:"<<endl;
cout<<student->id<<endl;
cout<<student->name<<endl;
cout<<student->school<<endl;
cout<<student->level<<endl;
cout<<student->group<<endl;
```

# Exercises

1. Given the following 3 students' records

| STUDENT 1 | | STUDENT 2 | | STUDENT 3 | |
|---|---|---|---|---|---|
| Student name | Mahoro Peace | Student name | Juru Ethar | Student name | Ineza Brianna |
| Age | 17 | Age | 17 | Age | 17 |
| Roll number | 201901 | Roll number | 201902 | Roll number | 201903 |
| Marks | 89.8 | Marks | 92.5 | Marks | 84.7 |

Using the concepts of structure and function call, write a program that accepts the data from the above tables and display them to the console as shown in the following screenshot.

```
201901, Mahoro Peace, 17, 89.80
201902, Juru Ethan, 17, 92.50
201903, Ineza Brianna, 17, 84.70
```

2. Build a structure that keep details of a date: The Year, month and the day of the month. Add a function to create the Date with Year, month and day as arguments.

3. Using structures get student data to the console and store them in a .txt file or a CSV file. Read stored data back to the console.

Rwanda Coding Academy

# Exercises(next)

4. Using structures get at least 5 student data from console and store them in a .txt file or a CSV file.

- Read stored data back to the console by sorting the students with their score using merge sort.
- The marks entered is out of 50. The program should not allow marks beyond 50 or below zero.
- The age should be between 10 and 30
- The rollnumber should be a valid positive integer.
- Display the average of marks entered with the number of distinct students recorded.